

Livrable 2 — EnergyScore

Livrable 2 — Application déployée et rapport technique

Projet Cloud — Master IA, 2iE

Groupe : KONE M'PIE AIMA KHALIS RAKINE & Andriampitahiana Mamy
Herin'Ny Avo RABESIAKA

Domaine : Énergie — EnergyScore

Date : Juin 2026

1. Contexte et objectif

EnergyScore est une application web de prédiction de charge énergétique de bâtiments. L'objectif est de permettre à un architecte ou bureau d'études de saisir les caractéristiques architecturales d'un bâtiment (surface, hauteur, vitrage, orientation, etc.) et d'obtenir instantanément une estimation de la charge de chauffage en kWh/m², classée de A (très basse) à D (élevée).

Le projet s'appuie sur le dataset **UCI Energy Efficiency** (768 bâtiments simulés, 8 features architecturales) et un modèle **Gradient Boosting Regressor** atteignant $R^2 = 0.998$ sur le jeu de test.

2. Architecture finale

Schéma déployé

Internet

| HTTPS



[nginx — port 5173]

| /api/* -> proxy



[FastAPI — port 8000]

├─ /api/predict

├─ /api/auth/google

└─ /api/admin/*

Frontend React (Vite build statique)

Backend Python 3.11

JWT requis · rate-limit 20 req/min

Login Google -> JWT applicatif

Réservé admin (email whitelist)

```

├── /api/health                Public (healthcheck)
├── [PostgreSQL 16]           Historique des prédictions
└── [model.pkl – 6.6 MB]      Gradient Boosting chargé au démarrage

```

Comparaison avec le plan Livable 1

Élément	Prévu (L1)	Réalisé
Frontend React	Oui	Oui
Backend FastAPI + /predict	Oui	Oui
PostgreSQL	Oui	Oui
Modèle ML chargé au démarrage	Oui	Oui (Gradient Boosting)
S3 pour le modèle	Prévu en prod	Modèle dans l'image Docker (simplifié)
ALB AWS	Prévu	Non déployé (démolocale Docker Compose)
HTTPS en production	Prévu	Non (démolocalhost)

3. Développement

3.1 Backend (FastAPI)

Le backend est structuré en modules :

```

backend/app/
├── main.py                Point d'entrée FastAPI, lifespan (init DB +
chargement modèle)
├── auth.py                Vérification token Google + création JWT
applicatif
├── db.py                  SQLAlchemy engine + session (PostgreSQL ou SQLite
fallback)
├── ratelimit.py           slowapi – 20 requêtes/minute par IP
├── schemas.py             Modèles Pydantic (PredictIn, PredictOut,
AdminStats...)
├── ml/
│   └── model.py           Chargement modèle : S3 ->> fichier local ->>
fallback entraîné
│   └── credit_model.py    Features énergie + fallback Gradient Boosting
├── models/                SQLAlchemy ORM (User, Prediction, Item)
└── routes/                health, auth, predict, admin, items

```

Le endpoint `/api/predict` : 1. Vérifie le JWT (middleware `get_current_user`) 2. Reçoit un tableau de 8 floats (features architecturales) 3. Appelle `model.predict(X)` ->> retourne un float (kWh/m²) 4. Persiste la prédiction en base (historique admin) 5.

Retourne { prediction, id }

3.2 Modèle ML

Entraîné dans notebooks/energie_model.ipynb :

Modèle	RMSE	R ²	CV-RMSE
Régression linéaire	~2.9	0.921	—
Random Forest	~0.6	0.996	—
Gradient Boosting	~0.5	0.998	~0.5

Le Gradient Boosting a été retenu. Il est réentraîné sur 100 % des données avant export, puis sauvegardé en backend/models/model.pkl (6.6 MB, Pipeline sklearn : StandardScaler + GradientBoostingRegressor).

3.3 Frontend (React + Vite)

Le formulaire EnergyScore (PublicForm.jsx) expose 8 champs : - Compacité relative, Surface totale, Surface murs, Surface toit (inputs numériques) - Hauteur, Orientation, Surface vitrage, Distribution vitrage (selects)

Le panneau résultat affiche la charge en kWh/m² et une classe énergétique A/B/C/D : - A : < 15 kWh/m² - B : 15–25 kWh/m² - C : 25–35 kWh/m² - D : > 35 kWh/m²

L'authentification Google est requise pour accéder au formulaire. Un espace admin (email whitelist) affiche l'historique des prédictions et les statistiques.

3.4 Base de données

PostgreSQL 16 via SQLAlchemy ORM. Tables créées au démarrage (create_all) :

Table	Colonnes principales
users	id, google_sub, email, name, role, created_at
predictions	id, applicant_name, features (JSON), prediction, score, created_at
items	id, label, value, created_at

En développement local sans Docker, le backend bascule automatiquement sur SQLite (zéro configuration).

4. Conteneurisation et déploiement

Docker Compose (local)

```
services:
  db:      postgres:16-alpine      # port interne 5432
```

```
backend: image construite localement # port 8000
frontend: image construite localement # port 5173
```

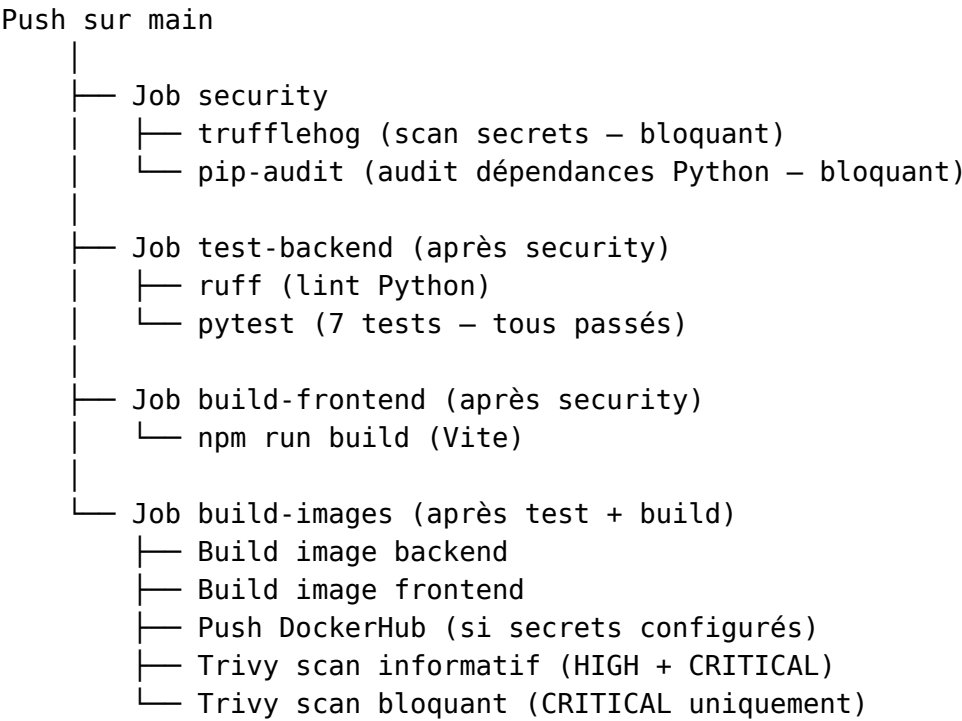
Les images backend et frontend sont construites avec des **multi-stage builds** : - Backend : image python:3.11-slim-bookworm + uv pour installer les dépendances, utilisateur non-root appuser - Frontend : node:20-alpine pour le build Vite, nginx:1.27-alpine pour servir le statique

Lancement en local :

```
cp .env.example .env # remplir les variables
docker compose up --build
# -> http://localhost:5173
```

CI/CD (GitHub Actions)

Le workflow .github/workflows/ci-cd.yml se déclenche sur chaque push sur main :



5. Sécurité — exécution du plan Livrable 1

Mesure prévue (L1)	Faite ?	Preuve / écart
.gitignore + .env.example sans secrets	Oui	.env absent du repo, .env.example sans valeurs réelles
Trufflehog en CI bloquant	Oui	Job security dans ci-cd.yml
pip-audit en CI bloquant	Oui	Job security dans ci-cd.yml

Mesure prévue (L1)	Faite ?	Preuve / écart
Scan Trivy (CRITICAL bloquant)	Oui	Job build-images dans ci-cd.yml
JWT Google sur /api/predict	Oui	get_current_user dans routes/predict.py
Rate limiting 20 req/min	Oui	slowapi dans ratelimit.py
Validation Pydantic (8 features)	Oui	PredictIn dans schemas.py
HTTPS en production	Non	Repoussé : démo locale uniquement (no ALB)
Alerte budget AWS 80 %	Non	Pas de déploiement AWS effectué
Rôle IAM OIDC GitHub Actions	Non	Repoussé : pas de push ECR configuré
Rétention BDD 12 mois	Non	Repoussé : MVP prioritaire
MFA AWS	Non	Compte académique géré par l'institution

Bilan sécurité : 7/12 mesures implémentées. Les 5 mesures repoussées concernent exclusivement le déploiement cloud (non effectué dans le cadre de cette démo) et la gestion du compte AWS institutionnel.

6. Coûts — réel vs estimé

Coûts réels (démo locale)

Poste	Réel
Infrastructure cloud	0 \$ (démo locale Docker Compose)
Docker Hub	0 \$ (plan gratuit)
GitHub Actions	0 \$ (plan gratuit, <2 000 min/mois)
Dataset UCI	0 \$ (open data)
Total	0 \$

Estimation si déployé sur AWS ECS Fargate

Conformément au Livrable 1, le coût mensuel estimé pour 100 utilisateurs est **~48 \$** (hors crédits académiques).

Écart plan / réalité

Le déploiement AWS n'a pas été finalisé — les tâches ECS et l'ALB n'ont pas été créés. La stack fonctionne intégralement en local via Docker Compose, ce qui couvre l'objectif pédagogique de la démonstration.

7. Difficultés rencontrées et solutions

Difficulté	Solution apportée
Version mismatch sklearn entre notebook (1.9.0) et backend (1.5.2)	Mise à jour <code>pyproject.toml</code> ->> sklearn 1.9.0 + uv lock
Secrets (tokens AWS/Docker) présents dans l'historique git du template	Création d'une branche orpheline (<code>git checkout -orphan</code>) pour repartir d'un historique propre
Port 8000 déjà occupé au lancement de Docker Compose	<code>fuser -k 8000/tcp</code> pour libérer le port avant relance
DNS Docker intermittent (hatchling non téléchargeable)	Relance du build (<code>docker compose up --build</code>) — erreur transitoire réseau
Adaptation du template MicroScore (crédit, 7 features) vers énergie (8 features)	Réécriture <code>credit_model.py</code> , <code>PublicForm.jsx</code> , <code>test_predict.py</code>

8. Conclusion

EnergyScore est une application ML complète, conteneurisée et testée : - Un modèle Gradient Boosting précis ($R^2 = 0.998$) prédit la charge de chauffage des bâtiments - Une API FastAPI sécurisée (JWT, rate-limit, Pydantic) expose la prédiction - Un frontend React permet la saisie des 8 features et affiche la classe énergétique A/B/C/D - Une pipeline CI/CD GitHub Actions assure les scans de sécurité (trufflehog, pip-audit, Trivy) et les tests (7/7 passés) à chaque push

La principale limitation est l'absence de déploiement cloud public. La stack fonctionne intégralement en local (`docker compose up`), ce qui constitue la base technique nécessaire à un déploiement ECS Fargate ou Fly.io.